

objects in your class rather than inheriting those objects' class. This is what we do when we add attributes to our classes. When designing your classes using inheritance, favor composition instead of inheritance. This is what we've been doing with all the examples in this and the previous chapter, as described in the following list:

- A `WizCoin` object “has an” amount of galleon, sickle, and knut coins.
- A `TTTBoard` object “has a” set of nine spaces.
- A `MiniBoard` object “is a” `TTTBoard` object, so it also “has a” set of nine spaces.
- A `HintBoard` object “is a” `TTTBoard` object, so it also “has a” set of nine spaces.

Let's return to our `WizCoin` class from the previous chapter. If we created a new `WizardCustomer` class to represent customers in the wizarding world, those customers would be carrying an amount of money, which we could represent through the `WizCoin` class. But there is no “is a” relationship between the two classes; a `WizardCustomer` object is not a kind of `WizCoin` object. If we used inheritance, it could create some awkward code:

```
import wizcoin

❶ class WizardCustomer(wizcoin.WizCoin):
    def __init__(self, name):
        self.name = name
        super().__init__(0, 0, 0)

wizard = WizardCustomer('Alice')
print(f'{wizard.name} has {wizard.value()} knuts worth of money.')
print(f'{wizard.name}'s coins weigh {wizard.weightInGrams()} grams.')
```

In this example, `WizardCustomer` inherits the methods of a `WizCoin` ❶ object, such as `value()` and `weightInGrams()`. Technically, a `WizardCustomer` that inherits from `WizCoin` can do all the same tasks that a `WizardCustomer` that includes a `WizCoin` object as an attribute can. But the `wizard.value()` and `wizard.weightInGrams()` method names are misleading: it seems like they would return the wizard's value and weight rather than the value and weight of the